**University of Bremen**
**Department of Computer Science**
**Prof. Dr. h.c. Michael Beetz, PhD**

**AICOR Institute for Artificial Intelligence**
**WiSe 2026/2027**
**Worksheet 3**

# Exercises for the Lecture Advanced Methods of AI

## Querying and Reasoning

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Task 1 on paper: 10 minutes in pairs, 5 minutes of discussion. Tasks 2 to 4: about 15 minutes in pairs each, then 5 to 10 minutes of discussion. They need a laptop with a web browser: open the notebook `03-pizza_queries.ipynb` on Binder (link on Stud.IP) and note your predictions and answers on this sheet.

**One expression, three readings.** Last week you wrote the concept ∃hasTopping.CheeseTopping. In EQL, the query language of KRROOD, the same thing is written

```
D = a(Pizza)(toppings=a(CheeseTopping)())
```

It is a **description**: it denotes a set of objects, as a concept does. It is a **question**: which objects of the knowledge base fit it? It is a **constraint**: a condition that an object has to satisfy, or must not satisfy. Today you use all three readings.

### Task 1: What Is a Query?

**The interpretation of Worksheet 2.** Four pizzas and their toppings; these are *all* the objects and *all* the hasTopping pairs. CheeseTopping, VegetableTopping and MeatTopping are disjoint.

| Pizza | Toppings |
|---|---|
| p1 | m1 : CheeseTopping, t1 : VegetableTopping |
| p2 | h2 : MeatTopping, m2 : CheeseTopping |
| p3 | none |
| p4 | v4 : VegetableTopping |

**(a) One description, several languages.** Write D as

i. a description logic concept,
ii. a first-order formula with one free variable x,
iii. an SQL query over the tables `pizza(id)` and `topping(id, pizza_id, kind)`, where `kind` is the name of the topping's class.

Which pizzas of the interpretation are the answer?

**(b) Reading EQL.** `pizzas` is the list of p1 to p4. What does each query return?

```
p = variable(Pizza, domain=pizzas)      # p ranges over the pizzas
t = flat_variable(p.toppings)           # t ranges over the toppings of p

Q1 = a(Pizza)(toppings=a(MeatTopping)()).from_(pizzas)
Q2 = entity(p).where(for_all(t, HasType(t, VegetableTopping)))
Q3 = set_of(p, t).where(HasType(t, CheeseTopping))
Q4 = the(Pizza)(toppings=a(VegetableTopping)()).from_(pizzas)
```

**(c) Question, description or constraint?** Four people in a pizzeria use D. For each: where do the candidate objects come from, and what counts as the answer?

i. The waiter, to the kitchen display: “Which pizzas on the counter have cheese?”

ii. The menu: “Cheesy pizza: a pizza with a cheese topping.”

iii. A guest: “A pizza with cheese, please.”

iv. The allergy sheet on the wall: “No order of a guest who avoids milk contains a pizza with cheese.”

**(d) True or entailed?** In Worksheet 2 you answered “does the KB entail φ?” with an argument about *every* model (Task 1b), and “which pizzas belong to C?” by looking at *one* interpretation (Task 2a). Which of the two does a query on p1 to p4 do? What does that mean for the question whether a pizza of which we only know a mozzarella topping is vegetarian?

## Task 2: Asking Questions

**Notebook, part 2.** Thursday evening in a pizzeria: seven pizzas on the counter, three orders, three guests. Run the cells from the top. For every **Predict**, fill in the sheet *before* you run the cell.

**(a) One question, two spellings.** Write `hot_pizzas` and `hot_orders`. american_hot_1 has two hot toppings. Why is it returned only once?

**(b) *a* or *the*?** Predict the result of “the pizza with fish” and of “the pizza with cheese”. A robot is told: “Bring me the pizza with cheese.” What should it do?

**(c) Some, only, none.** Predict with a cross which pizzas each query returns. Then write `vegetarian` (no meat and no fish topping).

| | margherita | napoletana | american_hot | mushroom | veggie | plain | unclosed |
|---|---|---|---|---|---|---|---|
| Q1 only vegetable toppings | | | | | | | |
| Q2 [a mozzarella, a tomato] | | | | | | | |
| Q3 no cheese topping | | | | | | | |

**(d) Answers are bindings.** Predict the number of rows of `hot_rows`. Then write `toppings_per_table`. What are the SQL words for what you used?

## Task 3: Reasoning with Functions, Predicates and Rules

**Notebook, part 3.** Ben avoids milk. No object says “napoletana_1 contains milk”; it has to be derived: mozzarella is a cheese, cheese contains milk, the pizza has that topping.

**(a) Functions.** Complete `allergens`. What is `is_vegetarian(p)` when p is a variable instead of a pizza?

**(b) Predicates and constraints.** Complete `IsSafeFor` and write `dangerous`. What does it return? Why is this query a constraint and not a question?

**(c) Rules.** Write the rules for fish and for gluten. How many warnings are derived? The predicate of (b) and the rules both find Ben's problem. When is each of them computed? Name one advantage of each.

**(d) Rules with exceptions.** Predict the label of each pizza. Which pizza gets none, and does that mean it is not vegetarian? After the ham on unclosed_1: which conclusion changed? What would an OWL reasoner have had to do?

| margherita | napoletana | american_hot | mushroom | veggie | plain | unclosed |
|---|---|---|---|---|---|---|
| | | | | | | |

## Task 4: Live Objects and Recorded Data

**Notebook, part 4.** So far the knowledge base was the *live* kitchen: Python objects that change while we look. At midnight the evening is *recorded* in a relational database (SQLite) and stays as it was.

**(a) The same question in SQL.** Why does the condition `topping.kind = 'CheeseTopping'` return nothing? What does the repaired query need that the EQL query did not?

**(b) The world moves on.** The cook grates parmesan over Ben's veggie_1 by mistake. Predict what each source says about cheese on veggie_1. Which one would have protected Ben?

| 1. query `cheesy` of Task 2, live objects | 2. the record | 3. milk warnings of Task 3(c) |
|---|---|---|
| | | |

**(c) What is missing?** Try to write `dangerous` in SQL over the record. What exactly is missing?

**(d) Four query languages.** Fill in the columns SQL and EQL from what you saw today. SPARQL (which you would use on pizza.owl) and Datalog/Prolog follow in the discussion.

| | SQL | SPARQL | Datalog / Prolog | EQL |
|---|---|---|---|---|
| What is queried? | | | | |
| From a pizza to its toppings | | | | |
| “A mozzarella is a cheese” comes from | | | | |
| “Not” means | | | | |
| Rules | | | | |
| Your own function in a query | | | | |

**(e) Your turn.** You are building a robot that tidies a kitchen. In words (or in EQL, with classes you invent), give one query of each kind and say what its domain is: a **question** the robot asks its knowledge base, a **description** of something that does not exist yet and that the robot has to produce, and a **constraint** that must never have an answer.